Lab 04 — EVM internals, Merkle proofs and a production-grade ERC-20

Khusanjon Ashuraliev · Introduction to Blockchain Programming · Óbuda University, NIK · November 2026

Item	Value
Repository	`https://github.com/KhusanAshuraliev/lab-04-evm-and-erc20`
Stack	Foundry 1.9.0, Solidity 0.8.24, EVM version Cancun, optimizer 200 runs. The only dependency is `forge-std` v1.16.2 for the tests. No OpenZeppelin, Solady or other token library: I wrote every contract in `src/` myself.
Evidence	Every output quoted below is saved in `evidence/`: test runs, traces, `forge inspect`, the gas report, coverage, Slither, and the transcripts of the bonus challenges.
Reproduce	`forge build && forge test`; `forge test --gas-report`; `slither . --exclude-dependencies`; `uv run airdrop/merkle.py build …` (E2); `bash script/e3_permit.sh` (E3)
Tests	**131 passed**: the course's three suites (65 tests), copied unchanged, plus 66 tests of my own. `forge coverage` of `src/`: **100 %** of lines, statements, branches and functions.

A note on gas figures. Foundry 1.9 runs tests with `isolate = true`, which executes each top-level call of a test as its own transaction. Gas measured around such a call therefore includes the 21,000 base cost of a transaction and its calldata. The course's expected values (for example 26,857 for the Solidity increment) were measured the same way, and my runs reproduce them exactly. Where I give an execution-only figure, I say so.

1. Storage layout of my token

```
$ forge inspect CampusToken storageLayout
| Name          | Type                                              | Slot | Offset | Bytes |
| _balances     | mapping(address => uint256)                       | 0    | 0      | 32    |
| _allowances   | mapping(address => mapping(address => uint256)) | 1    | 0      | 32    |
| _totalSupply | uint256                                           | 2    | 0      | 32    |
| _name         | string                                            | 3    | 0      | 32    |
| _symbol       | string                                            | 4    | 0      | 32    |
| nonces        | mapping(address => uint256)                       | 5    | 0      | 32    |
| _members      | mapping(bytes32 => mapping(address => bool))    | 6    | 0      | 32    |
| _admins       | mapping(bytes32 => bytes32)                       | 7    | 0      | 32    |
| treasury      | address                                           | 8    | 0      | 20    |
| feeBps        | uint16                                            | 8    | 20     | 2     |
| paused        | bool                                              | 8    | 22     | 1     |
| claimRoot     | bytes32                                           | 9    | 0      | 32    |
| hasClaimed    | mapping(address => bool)                          | 10   | 0      | 32    |
```

The layout follows the inheritance order `ERC20 → ERC20Permit → AccessControl → CampusToken`, and within each contract the declaration order. Eleven slots (0–10) are used. The cap, the cached domain separator and the hashed version are `immutable`, so they live in the runtime code and have no row. The role ids and `MAX_FEE_BPS` are `constant` and have no row either.

The one packed slot is slot 8. `treasury` (20 bytes, offset 0), `feeBps` (2 bytes, offset 20) and `paused` (1 byte, offset 22) use 23 of the 32 bytes; bytes 23–31 are empty. I placed these three next to each other on purpose. `_update` reads all three on every transfer, so one cold `SLOAD` (2,100 gas) serves the whole check, where three separate slots would cost three. `test_PackedConfigurationSlot` sets the fee to 123 and pauses the token, then splits the raw word: bits 0–159 are the treasury, bits 160–175 are 123, bit 176 is 1, and everything above is zero. Every other slot holds one full word or is the anchor of a mapping or a string.

Slot arithmetic for one mapping entry. I declared `_balances` as the first variable, so it is slot 0. The balance of an account *a* is at

slot(a) = keccak256(abi.encode(a, uint256(0))) — the address padded to 32 bytes, then the slot number

`test_BalanceSlotArithmetic` computes this slot for Alice, reads it with `vm.load` and gets 1000×10^{18}, which equals `balanceOf(alice)`. Slot 0 itself reads zero (a mapping's own slot stays empty), and slot 2 reads exactly `totalSupply()`.

The specimen contract. `forge inspect StorageLayout storageLayout` shows `packedA`, `packedB`, `packedC` in slot 0 at offsets 0, 16, 24; `wide` alone in slot 1; `owner` and `flag` in slot 2 at offsets 0 and 20; the mapping, the array and the nested mapping in slots 3, 4, 5; `_notReallySecret` in slot 6. There is no row for `CONSTANT_VALUE` or `IMMUTABLE_VALUE`. All 10 tests of `StorageLayoutTest` pass. `test_PrivateStorageIsPubliclyReadable` reads the "private" value back with one `vm.load`, which on a live chain is one `eth_getStorageAt`. The packing test logs **89,134** gas for the bad order (3 slots) and **66,959** for the good order (2 slots). The difference of 22,175 gas is one zero-to-nonzero `SSTORE` (22,100) plus a little bookkeeping.

2. Solidity versus Yul: gas and the differential tests

I wrote two Yul functions of my own besides the course's `incrementYul`, `sumYul` and `guardYul`.

- **`MerkleYul.verifyYul`** is a complete Merkle-proof verifier in assembly. It walks the proof straight from calldata and hashes each pair in the scratch space 0x00–0x3f. It orders the pair without a branch: with x = node ⊕ sibling, lo = node ⊕ (x · [sibling < node]) and hi = x ⊕ lo. The Solidity reference is my `MerkleProof.verify`, which the token uses.
- **`YulCounter.sumCheckedYul`** sums a calldata array and, on overflow, reverts with exactly the bytes that Solidity's checked arithmetic produces: `Panic(uint256)`, selector 0x4e487b71, code 0x11. So it must agree with `sumSolidity` on *every* input, including the ones that revert.

Function	Solidity gas	Yul gas	Saved	Where the saving comes from
increment (warm slot)	26,857	26,799	58 (0.2 %)	one overflow check; the SSTORE (2,900) and the transaction (21,000) dominate
checked sum of 100 words	31,664	23,391	8,273 (26 %)	per element Solidity does a bounds check, a checked add and a checked i++; Yul does one lt
Merkle verify, 14 levels	10,461	9,310	1,151 (11 %)	no bounds checks, no branch for the pair order, no memory copy of the proof

Differential tests (`test/YulDifferential.t.sol` and the course's `YulCounterTest`), 512 runs each:

- `testFuzz_CheckedSumAgreesIncludingReverts(uint256[])` calls both versions with low-level `staticcall`s and asserts the same success flag and the **same return data or the same revert data**. With full 256-bit inputs, overflow is common, and in that case it also asserts that the bytes equal `stdError.arithmeticError`.
- `testFuzz_MerkleAgreesOnRealTrees` builds a random tree of $2^0 \ldots 2^8$ leaves, checks that both versions accept the right proof for a random leaf, flips one bit of one proof element, and checks that both reject it.
- `testFuzz_MerkleAgreesOnGarbage` compares both on arbitrary words. It then recomputes the root from the same garbage, so that the "true" path is fuzzed as well as the "false" path. `test_MerkleEqualSiblings` covers the corner case lo = hi.
- `testFuzz_SumImplementationsAgree` and `testFuzz_GuardImplementationsAgree` (course) compare the unchecked sum and the custom-error revert.

The two big savings are on loops over calldata, where the compiler repeats its checks for every element. The increment shows the other side: when the cost is one storage write, assembly saves almost nothing and

removes a check the reader must now verify by hand. I use assembly in the token only for hashing in the scratch space, and in the proxy for forwarding return data of unknown length.

3. The proxy storage collision, and why it is silent

```
$ forge test --match-test test_BrokenProxyFailsSilentlyAfterCollision -vvvvv
  ├─ new BrokenProxy@0xF62849F9…
  │    ├─ storage changes:
  │    │   @ 0: 0 → 0x…5615deb798bb3e4dfa0139dfa1b3d433cc23b72f        (the address of LogicV1)
  ├─ [30313] BrokenProxy::fallback(57005)
  │    ├─ [4208] LogicV1::setValue(57005) [delegatecall]
  │    │    ├─ emit ValueSet(value: 57005)
  │    │    ├─ storage changes:
  │    │    │   @ 0: 0x…5615deb798bb3e4dfa0139dfa1b3d433cc23b72f → 57005   (0xDEAD)
  │    └─ ← [Return]
  ├─ [25950] BrokenProxy::fallback()
  │    ├─ [0] 0x000000000000000000000000000000000000dEaD::value() [delegatecall]
  │    │    └─ ← [Stop]
  │    └─ ← [Return]
```

What happens. `BrokenProxy` keeps `implementation` in slot 0, and `LogicV1` keeps `value` in slot 0. A `delegatecall` runs LogicV1's code against the *proxy's* storage, so an ordinary `setValue(0xDEAD)` succeeds and writes 57005 = 0xDEAD into slot 0 of the proxy, right over the implementation address. The next call is delegated to address 0xDEAD, which has no code. The trace shows `[0]` gas used, `Stop`, and the proxy returns successfully with **empty data**. Nothing reverts anywhere in the trace. My extra test `test_BrokenProxyReturnsNothingAfterCollision` shows that `value()` now returns zero bytes, and that a further `setValue(1)` also "succeeds" while slot 0 stays 0xDEAD: nothing is written.

Why it is silent and not a revert, in two halves:

1. **The EVM treats a call to an address without code as a successful no-op.** Executing empty code is the same as executing `STOP`: the call returns success = 1 with no return data. This is how plain ETH transfers to an EOA work, so it cannot be an error.
2. **The protection Solidity normally adds does not apply here.** For a high-level call such as `LogicV1(x).value()`, Solidity checks `extcodesize(x) > 0` before the call and reverts otherwise. The proxy issues its `delegatecall` from inline assembly, where no such check exists, and it forwards whatever came back (nothing) as a success.

The only place where anything fails is in a caller that tries to decode a `uint256` from zero bytes, and that caller does not know why. The function that could repair the pointer was in the code that is no longer reachable, so the proxy is bricked for good, which is the shape of the Parity incident.

The fix (`SafeProxy`) keeps the implementation and the admin at the EIP-1967 slots `keccak256("eip1967.proxy.implementation") − 1` = 0x3608…2bbc and `keccak256("eip1967.proxy.admin") − 1` = 0xb531…6103. Both tests assert these constants. No variable, mapping entry or array element can land there. `test_SafeProxyAvoidsCollision` writes 0xDEAD to slot 0 and the pointer survives. `test_UpgradePreservesState` reads a value stored under V1 through V2 (21, then 42 after the multiplier). `upgradeTo`, and in my version also the constructor, refuse an address without code (`NotAContract`), which turns the silent failure above into a revert. All 8 course tests of `ProxyTest` and my 4 in `ProxyExtraTest` pass.

4. The approve race

`test_ApproveRaceCondition` passes; the trace (`evidence/part_c_approve_race.txt`) shows the calls in this order and ends with `balanceOf(bob) = 150 × 10`^{18}:

1. Alice calls `approve(Bob, 100)`. It is mined; the allowance is 100.

2. Alice changes her mind and broadcasts `approve(Bob, 50)`. It waits in the mempool.
3. Bob sees it in the mempool and front-runs it with `transferFrom(Alice, Bob, 100)` at a higher fee. His transaction is mined first; he receives 100 and the allowance drops to 0.
4. Alice's transaction is mined. It does not reduce anything: it *sets* the allowance to 50.
5. Bob calls `transferFrom(Alice, Bob, 50)`. In total he has taken 150 from an owner who never meant to allow more than 100.

Root cause. `approve` overwrites the allowance with a new value; it cannot express "only if it is still 100". Between Alice's decision and the moment her transaction runs, the old allowance can be used up, and the new value is then granted on top.

Mitigations. Approve to 0 first, wait until that is mined, check how much was spent, and then approve the new amount. Better, use `permit`: every authorisation is signed for one specific nonce and is consumed when used, so an old authorisation can never stack with a new one. The race is part of the ERC-20 interface, so I did not "fix" it inside `approve`; the test demonstrates it.

5. Merkle allowlist against a mapping, for 10,000 recipients

I measured both approaches in `test/Airdrop.t.sol`. For the mapping I wrote `MappingAllowlist.setMany`, which stores one allocation per recipient, and called it once with 10,000 entries. For the Merkle approach I set the root produced by my tool (bonus E2), and I claimed with real 14-element proofs.

Approach	Deployer pays (measured)	Claimant pays (measured)
`mapping(address => uint256)`, 10,000 entries	**238,763,639 gas** (23,876 per entry)	83,887 per claim
Merkle root, 10,000 leaves	**47,747 gas** for `setClaimRoot` (26,747 without the 21,000 base)	88,901–108,275 per claim, median 90,413

The deployer's side. The mapping needs one zero-to-nonzero `SSTORE` (22,100) per recipient, plus loop and calldata costs: 238.8 million gas. That is **eight full 30-million-gas blocks**, so it cannot even be sent as one transaction; it would have to be split into at least eight, and it would cost the deployer about 5,000 times as much as the root. The root is one `SSTORE` plus the role check and the event.

The claimant's side. A Merkle claim costs about 6,500 gas more than a mapping claim, because the claimant sends the 14 proof words (448 bytes of calldata, about 7,000 gas) and hashes 14 pairs. The hashing itself is small: about 14 × 70 ≈ 1,000 gas, roughly 1 % of the claim. Two fresh storage writes dominate: `hasClaimed[msg.sender] = true` (22,100) and the claimant's balance going from zero to non-zero (22,100). Then come the 21,000 base cost and the update of `totalSupply`. The most expensive claim, 108,275, was the very first one, where `totalSupply` itself went from 0 to non-zero (another ~17,000). With the course's 2-element proofs a claim costs 64,941. In short, the Merkle root moves almost the whole cost from one deployer to the recipients who actually claim, and each of them pays only a little more than with a mapping.

6. Invariants

I ran two invariant suites: the course's `CampusTokenInvariantTest`, unchanged, and my own `CampusTokenModelTest`. My handler drives eight actions on five actors: transfer, transferFrom with a partial allowance, a permit followed by a spend, mint, burn, claim, a fee change, and moving the treasury to another actor. It also keeps an independent **model** of every balance, updated with my own fee arithmetic. Both suites ran 64 runs × 32 calls = 2,048 calls each with **0 reverts**, so every call did real work.

```
CampusTokenInvariantTest invariants (runs: 64, calls: 2048, reverts: 0)
[PASS] invariant_FeeStaysWithinTheHardLimit      [PASS] invariant_SumOfBalancesEqualsTotalSupply
[PASS] invariant_SupplyMatchesMintMinusBurn      [PASS] invariant_TotalSupplyNeverExceedsCap
CampusTokenModelTest invariants (runs: 64, calls: 2048, reverts: 0)
[PASS] invariant_BalancesMatchTheModel           [PASS] invariant_FeeWithinLimit
[PASS] invariant_NoncesAndClaimsAreExact         [PASS] invariant_SumOfBalancesIsSupply
[PASS] invariant_SupplyIsMintedMinusBurned       [PASS] invariant_SupplyWithinCap
```

Invariant, in one sentence	A failure would mean
Sum of balances = total supply: the balances of all holders add up to `totalSupply`.	a balance changed without the supply, or the other way round: tokens appeared or vanished outside the accounting, typically in the fee path
Total supply = minted – burned: the supply can be rebuilt from the history of mints (including claims) and burns alone.	a hidden mint or burn path: some code creates or destroys tokens without going through `mint`, `claim` or `burn`
Total supply ≤ cap.	a mint path that skips the cap check
Fee ≤ hard maximum: `feeBps` never exceeds 500.	a configuration path around `setFeeBps`; an admin could raise the fee and confiscate transfers
Every balance equals my model (mine).	tokens went to the wrong account even though the totals are right, for example a fee rounded the wrong way or sent to the old treasury
Nonces and claims are exact (mine): `nonces(a)` counts the permits *a* signed, and `hasClaimed(a)` is true exactly for accounts that claimed.	a signature could be used twice or not at all, or an allocation could be claimed twice

Do they catch a real bug? In a scratch copy I planted the classic fee-on-transfer mistake: the fee was *minted* to the treasury instead of moved from the sender (`evidence/invariants_catch_planted_bug.txt`). Five invariants failed at once: "supply = minted – burned", "supply ≤ cap" (in both suites) and "balances match the model". The interesting part is which one did **not** fail: **sum of balances = total supply** kept passing, because a hidden mint raises the balances and the supply by the same amount. The ghost counters of minted and burned tokens, kept outside the contract, are what catch it. This is why I count "supply = minted – burned" as the most important invariant of a fee token.

7. Slither

```
$ slither . --exclude-dependencies          # Slither 0.11.4, 100 detectors
INFO:Slither:. analyzed (21 contracts with 100 detectors), 21 result(s) found
```

The run with `--filter-paths "test|script|lib"` gives the same 21 results. **None of them is in `CampusToken.sol` or in `src/lib/`.** In a first run there was one: `uninitialized-local` for `uint256 fee;` in `_update`, which relied on the default zero on purpose. I wrote `uint256 fee = 0;` to make the intent explicit and the finding went away. My triage of the 21:

Detector (count)	Where	Triage
`assembly` (10)	YulCounter ×4, MerkleYul, Forwarder, SafeProxy ×2, SafeProxyWithInit, StorageLayout.readSlot	Intended. The Yul functions exist to be compared with Solidity and are covered by differential tests. The proxy needs assembly to forward return data of unknown length and to use the EIP-1967 slots. `readSlot` demonstrates reading a private slot. All blocks except the forwarding ones are marked `memory-safe`.
`missing-zero-check` (3)	BrokenProxy constructor, SafeProxyWithInit constructor, StorageLayout.setOwnerAndFlag	BrokenProxy is the deliberate bug. For SafeProxyWithInit it is a **false positive**: the base constructor `SafeProxy(impl, …)` runs first and reverts with `NotAContract` for any address without code, the zero address included, before the `delegatecall` is reached. The setter of the specimen has no meaning to protect.
`naming-convention` (3)	`__gap` ×2, `IMMUTABLE_VALUE`	Intended. `__gap` is the established name of a storage gap, which upgrade tools look for. `IMMUTABLE_VALUE` is the name the course tests call.
`locked-ether` (1)	BrokenProxy	True for the broken proxy after the collision (its fallback is payable and the code it forwards to is gone), and part of what it demonstrates. `SafeProxy` is not reported: plain ETH is forwarded to the logic, which refuses it (`test_ProxyRefusesPlainEth`).
`low-level-calls` (1)	SafeProxyWithInit constructor	Intended: a `delegatecall` of the initializer during construction, so that no one can front-run `initialize`. Its failure is checked and the revert data is bubbled up (`test_RevertWhen_InitializedWithZeroCap`).
`unindexed-event-address` (1)	SafeProxy.AdminChanged	Intended: EIP-1967 defines `AdminChanged(address previousAdmin, address newAdmin)` without indexed fields, and explorers decode it by that exact signature.
`constable-states` (1), `immutable-states` (1)	UpgradeableBase.baseValue, BrokenProxy.implementation	False positives for what they demonstrate. A constant or immutable would occupy no slot, and the whole point of both variables is that they occupy one.

What Slither cannot find. Slither matches code patterns, not intent. It would not notice the approve race, because that is the specified behaviour of ERC-20. It would not notice a fee that is correct but economically wrong (say 5 % on a stablecoin), a wrong but well-formed constant, or a mint that is allowed but should not be. The wrong n/2 constant in the reference `ECDSA.sol` (section 12) is exactly such a bug: the constant is well-formed, and only knowing what it *should* be reveals the error. A fuzz test with random keys found it on its first run.

8. Three vulnerabilities my contract does not have

1. Signature forgery through ecrecover's failure modes (malleability and the zero address). `ecrecover` accepts both (r, s) and (r, n − s) for the same message, and returns `address(0)` instead of reverting for a malformed signature. `ECDSA.recover` (`src/lib/ECDSA.sol`), which `permit` calls, first rejects every s > n/2 with `InvalidSignatureS` and then rejects a zero result with `InvalidSignature`. Without the second check, `permit(address(0), spender, …)` with garbage bytes would "prove" that the zero address signed. Tests: `test_RevertWhen_PermitUsesHighS`, `test_RevertWhen_SignatureIsGarbage`, and `testFuzz_PermitFromAnyKey`, which checks over 512 random keys that the honest signature passes and its high-s twin fails.

2. Permit replay: the same signature twice, on another chain, or through another proxy. In `ERC20Permit.permit` the line `uint256 nonce = nonces[owner]++;` builds the digest with the current nonce and uses it up in the same statement, so a second submission is checked against nonce + 1 and recovers an unrelated address. `DOMAIN_SEPARATOR()` returns the cached value only while `block.chainid` and `address(this)` both equal their values at deployment; otherwise it rebuilds the separator with the current ones. A signature therefore binds to one chain and one token address. Tests: `test_RevertWhen_PermitReplayed`, `test_RevertWhen_PermitReplayedOnAnotherChain`, `test_DomainSeparatorDependsOnChainId`, `test_DomainSeparatorUsesTheProxyAddress`; on anvil, step 8 of bonus E3.

3. Cap bypass and double claim through the airdrop. `claim` (`src/CampusToken.sol`) has no balance logic of its own. It sets `hasClaimed[msg.sender] = true` *before* it calls `_mint`, and `_mint` goes through `CampusToken._update`. There, `if (newSupply > cap_) revert CapExceeded(…)` is the only cap check, shared by `mint` and `claim`. A second claim stops at `AlreadyClaimed`. A claim that would cross the cap reverts with the same error as an over-cap mint, and the revert also undoes the `hasClaimed` write. Tests: `test_RevertWhen_ClaimedTwice`, `test_ClaimHitsTheCapWithCapExceeded`, `test_RevertWhen_ClaimIsPaused` (which also asserts that a failed claim is not recorded), and the invariant "total supply ≤ cap" with claims in the handler.

9. Bonus E2 — a real-sized airdrop

`airdrop/merkle.py` (Python, run with `uv`; keccak-256 from `pycryptodome`, because `hashlib.sha3_256` is the NIST variant with different padding) reads a CSV of `address,amount` rows and writes the root and every proof to JSON. It uses exactly the contract's encoding: leaf = keccak256(keccak256(abi.encode(account, amount))), and each pair is hashed in ascending order. When a level has an odd number of nodes, I pair the last node with `bytes32(0)`. Zero is not a hash anyone can produce, so the padding can never be claimed, and every proof of a 10,000-leaf tree has exactly $\lceil \log_2 10{,}000 \rceil = 14$ elements. The tool refuses a CSV with duplicate addresses, and before writing it checks every proof against the root.

```
$ uv run airdrop/merkle.py generate airdrop/recipients.csv 10000
$ uv run airdrop/merkle.py build airdrop/recipients.csv airdrop/proofs.json
recipients : 10000
root       : 0xfb162494eb295d5959948596d8fe4bad6cb720537cb98a6657b710525c3c129b
proof size : 14 elements for every recipient (ceil(log2 10000))
checked    : all 10000 proofs verify against the root in Python
time       : 0.62 s
```

`test/Airdrop.t.sol` loads the 11.5 MB JSON with `vm.readFile` and `vm.parseJson*`, deploys the token with that root, and claims for entries 0 (first), 9,999 (last) and 9,159 (random). For each it asserts 14 proof elements, that `leafFor` in Solidity equals the leaf Python computed, the minted amount, and `hasClaimed`. `test_RealProofGuards` then shows that a real proof fails with amount + 1, fails for another sender, and fails a second time. The measured claim gas, 88,901–108,275, is compared with the mapping in section 5: the table

in the lab sheet estimates "~14 hashes" for the claimant, and my measurement shows that the hashes are only about 1 % of what a claimant actually pays.

10. Bonus E3 — permit from a real signature

`script/e3_permit.sh` starts `anvil` and deploys the token with `forge script script/Deploy.s.sol --broadcast` (development key #0). It mints 1,000 CMPS to the owner (key #1) and signs this typed data **outside Solidity** with `cast wallet sign --data --from-file`:

```
{ "types": {
    "EIP712Domain": [ {"name":"name","type":"string"}, {"name":"version","type":"string"},
                      {"name":"chainId","type":"uint256"}, {"name":"verifyingContract","type":"address"} ],
    "Permit": [ {"name":"owner","type":"address"}, {"name":"spender","type":"address"},
                {"name":"value","type":"uint256"}, {"name":"nonce","type":"uint256"},
                {"name":"deadline","type":"uint256"} ] },
  "primaryType": "Permit",
  "domain":  { "name": "Campus Token", "version": "1", "chainId": 31337,
               "verifyingContract": "0x5FbDB2315678afecb367f032d93F642f64180aa3" },
  "message": { "owner": "0x70997970C51812dc3A010C7d01b50e0d17dc79C8",
               "spender": "0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC",
               "value": "250000000000000000000", "nonce": 0, "deadline": 1793891262 } }
```

The script splits the 65-byte signature into r, s and v, and the **spender** (key #2) submits it with `cast send … "permit(address,address,uint256,uint256,uint8,bytes32,bytes32)"`. Results (`evidence/e3_permit.txt`):

- the `permit` transaction has **status 1 (success)**, 74,560 gas, paid by the spender;
- `cast call … allowance(owner, spender)` returns **250000000000000000000**, the signed value, and `nonces(owner)` returns **1**; the owner has sent **0** transactions;
- resubmitting the same signature **reverts** with `InvalidPermitSigner(0x3416…8e5b, owner)`: the digest is now built with nonce 1 and recovers an unrelated address;
- the spender then moves 100 CMPS with `transferFrom`, leaving an allowance of 150.

Which field I got wrong first. Honestly, none on my first run. I had the troubleshooting list of the lab sheet in front of me, so I took the domain name from `name()`, version "1", chain id 31337 from `cast chain-id`, and the address the deployment actually produced. I also wrote `value` as a decimal string, because 250×10^{18} is larger than any number JSON tools represent exactly. To see what a wrong field looks like, the script then signs the same message with version "1.0". That `permit` reverts with `InvalidPermitSigner(0xA55A…3e4b, owner)`. The contract cannot tell *which* field differs. It only sees that the signature recovers to some other address, which is why one wrong character anywhere in the domain looks the same as a wrong key.

11. Bonus E4 — an upgradeable token

`CampusTokenUpgradeable` inherits all of `CampusToken` and runs behind my EIP-1967 `SafeProxy`. It replaces the constructor with `initialize`, which may run only once (`AlreadyInitialized`), and it keeps the cap in storage, because an immutable lives in the implementation's code and would be shared by every proxy. The implementation's own constructor marks the implementation as initialised, so nobody can initialise it directly and make themselves its admin. `SafeProxyWithInit` runs `initialize` inside the proxy's constructor, so there is never a moment when the proxy exists uninitialised. Behind the proxy `address(this)` is the proxy, so `DOMAIN_SEPARATOR()` is rebuilt with the proxy's address and name. `CampusTokenV2` adds one state variable, `maxTransfer`, and one function, `setMaxTransfer` (CONFIG_ROLE), which limits ordinary transfers but not mint or burn.

```
variable      V1 slot/offset  V2 slot/offset  check           (forge inspect, evidence/e4_layout_v1_v2.txt)
_balances…hasClaimed   0 … 10          0 … 10  unchanged
_storedCap               11/0            11/0  unchanged
_initialized             12/0            12/0  unchanged
__gap (uint256[48])     13/0            13/0  unchanged   (slots 13–60)
maxTransfer                –            61/0  NEW, appended
no existing slot changed: True
```

The 13 tests in `test/Upgradeable.t.sol` pass. Some highlights:

- `test_UpgradeKeepsAllState` creates balances, a fee payment to the treasury, an `approve`, a `permit` (nonce 1), an extra minter and a claim, then upgrades. Afterwards all balances, the supply (1,505 CMPS), both allowances, the nonce, all roles, the fee, the treasury, the cap, `hasClaimed`, the name and the domain separator are unchanged. A permit signed for nonce 1 works after the upgrade, and a second `initialize` still reverts.
- `test_RevertWhen_UpgradeByNonAdmin` shows that even the token's admin is refused with `NotAdmin`: only the proxy admin may upgrade.
- `test_V2AppendsItsVariable` reads slot 11 (the cap) and slot 61 (`maxTransfer` = 777) on the live proxy.
- `test_V2NewFunction` shows that the new limit and its role check work.

What would break if V2 inserted the variable at the top. I tested it instead of only describing it. `CampusTokenV2Wrong` puts `maxTransfer` in a base contract that comes first in the inheritance order. `forge inspect` shows it in slot 0, and every old variable moved down one slot. After the upgrade (`test_PrependingAVariableCorruptsEverything`):

- `balanceOf(alice)` reads 0, because balances are now looked up from slot 1, where the allowances live;
- `totalSupply` reads slot 3, which holds the encoded string "Campus Token";
- the admin has lost `DEFAULT_ADMIN_ROLE`, because the role table is now looked up from slot 7, where the role admins used to be;
- `cap()` reads slot 12, which held the `_initialized` flag, so the cap is now **1 wei** and every mint reverts.

Nothing reverts during the upgrade itself. The new code simply reads the old storage with the wrong names, the same silence as in section 3.

12. A bug I found in the reference code

While writing my `ECDSA.sol` I computed n/2 for secp256k1 myself and compared it with the reference: `0x7FFF…681B20A` has **63** hex digits, while n/2 = `0x7FFF…681B20A0` has 64. The reference constant is therefore n/32, and `recover` rejects every honest signature with s between n/32 and n/2 — about 15 out of 16. The course suite does not notice, because all of its permit tests sign the same digest with the same key, whose s happens to be small. A fuzz test that signs with random keys fails on its first run against the reference (`evidence/reference_ecdsa_constant.txt`). My constant is correct. `testFuzz_PermitFromAnyKey` passes for 512 random keys, and `test_HonestSignaturesUseTheWholeLowerHalf` shows that most honest signatures do have s > n/32.

13. Requirements and tests

#	Requirement	Evidence that it is done
1	ERC-20, single funnel	`grep` over `src/` finds the only four writes to `_balances` and `_totalSupply` at lines 125–142 of `ERC20.sol`, all inside `_update`. `Transfer` is emitted for mint (from 0), burn (to 0), transfers and both legs of a fee transfer; `Approval` for `approve` and `permit`. A transfer to address 0 reverts with `InvalidReceiver`, an overdraft with `InsufficientBalance`.
2	Cap and roles	Minting past the cap reverts through `mint` and `claim` (`CapExceeded`); burning frees headroom. The deployer holds none of the four roles. MINTER, PAUSER and CONFIG on three different accounts can each do only their own job; non-holders get `MissingRole`; a revoked minter cannot mint.
3	Transfer fee	1,000 at 1 % gives 990 + 10; mint, burn and treasury transfers pay none; the supply is unchanged by any transfer; 501 bps is rejected with `FeeTooHigh(501, 500)`; 3 wei at 2.5 % succeeds without a fee; `testFuzz_FeeSplit`.
4	Permit	A valid permit sets the allowance and moves the nonce to 1. Expired (`PermitExpired`), replayed and wrong-signer permits revert (`InvalidPermitSigner`), as do high s (`InvalidSignatureS`) and garbage (`InvalidSignature`). `DOMAIN_SEPARATOR()` changes after `vm.chainId` and equals the EIP-712 formula before and after.
5	Merkle claim	One valid claim mints exactly the leaf's amount. A second claim (`AlreadyClaimed`), another account's proof, a wrong amount, an internal node as a leaf, and the second-preimage "account" built from two leaves all revert (`InvalidMerkleProof`). Real 14-element proofs come from E2.
6	Yul	Section 2: two Yul functions of my own. Differential fuzzing over 512 runs compares return data *and* revert data. Gas: 31,664 → 23,391 and 10,461 → 9,310.
7	Proxy collision and EIP-1967	Section 3. Broken proxy: success with empty data after the collision. Safe proxy: pointer kept, V1 state readable under V2, an upgrade to a non-contract reverts.
8	Tests	Every custom error of my contracts is triggered by at least one test. `forge coverage` of `src/`: 100 % lines (326/326), statements (303/303), branches (46/46), functions (95/95). Fuzz tests: 5 from the course and 6 of mine, 512 runs each. Invariants: 10 across two suites, 0 reverts in 4,096 calls.

```
$ forge test --match-path "test/{CampusToken,CampusTokenInvariant,EvmInternals}.t.sol"     # the course suite, unchanged
Ran 5 test suites in 263.22ms (352.41ms CPU time): 65 tests passed, 0 failed, 0 skipped (65 total tests)

$ forge test
Ran 11 test suites in 1.72s (3.69s CPU time): 131 tests passed, 0 failed, 0 skipped (131 total tests)
```